

```

system nano_vllm_cuTile_Inference_v1 {

    // =====
    // 1. Design Space (아키텍처 결정 공간)
    // =====
    design_space {
        // 파이프라인 모드
        inference_phase: ["prefill", "decode"]

        // 커널 퓨전 수준
        fusion_policy: ["full_fusion", "attention_only"] // Prefill/Decode 모두 QKV->Attn->0

        // KV 캐시 정책
        kv_cache_mode: ["paged", "contiguous"] // PagedAttention 스타일 혹은 연속

        // 런치 오버헤드 제거 전략
        execution_mode: ["cuda_graph", "eager"] // CUDA Graph를 사용한 일괄 실행 vs

        // SM 파티셔닝 (멀티컨텍스트 서빙 시)
        sm_partitioning: ["none", "green_contexts"] // Prefill/Decode 전용 SM 그룹 분리
        prefill_sm_ratio: [0.5, 0.75, 0.875] // 전체 SM 중 Prefill에 할당할 비율
    }

    // =====
    // 2. Tuning Space (미세 조정 파라미터)
    // =====
    tuning_space {
        // Prefill FMHA 타일링 (cuTile 기반)
        prefill_tile_m: [64, 128] // M 방향 타일 (128은 레지스터 제한 주의)
        prefill_tile_n: [64, 128]

        // Decode 어텐션 (GEMV 스타일)
        decode_kv_block_size: [16, 32] // KV Cache 읽기 단위 (벡터화)
        decode_split_cta: [1, 2, 4] // 여러 CTA로 partial softmax 수행 (FlashDecoding)

        // 공통 연산 정밀도
        softmax_scheme: ["online", "naive"]
        accum_dtype: ["f32", "f16"]

        // CUDA Graph 사용 시 고정 버퍼 크기
        max_prompt_len: [512, 1024, 2048] // Prefill용 고정 크기 (가변 입력 대응)
        max_seq_len: [512, 1024, 2048, 4096] // KV Cache 총 길이
    }

    // =====
    // 3. 모델 및 지식 베이스 (Know-How)
    // =====
    model {
        type PrefillConfig matches { tile_m: int, tile_n: int, fusion: bool, graph: bool }
        type DecodeConfig matches { kv_block: int, split_cta: int, graph: bool }
        state current_bottleneck: "Launch_Overhead" | "SM_Starvation" | "Memory_BW" | "Register_Files"
    }

    knowledge {

```

```

// (1) Python 런치 오버헤드 경고
fact python_launch_overhead(t: TracelItem.Performance) {
    assert: "만약 execution_mode == 'eager'이고 전체 커널 런치 횟수 > (레이어 수 * 3)
    action: "execution_mode를 'cuda_graph'로 강제 전환하고, 디코딩 루프 전체를 하나의
}

// (2) Prefill 시 SM 기아 방지 (작은 배치)
fact prefill_sm_utilization(t: TracelItem.Analysis) {
    assert: "배치 크기 == 1이고 헤드 수 < SM 수의 2배이며, tile_m * tile_n이 작으면 생
    action: "fusion_policy를 'full_fusion'으로 설정하여 단일 커널의 작업량을 늘리고,
}

// (3) Decode 시 어텐션 방식 자동 선택
fact decode_attention_style(t: TracelItem.SequenceLength) {
    assert: "Q_len == 1일 때 full FMHA 커널을 사용하면 연산량이  $O(N^2)$ 으로 불필요하게
    action: "GEMV 스타일의 FlashDecoding 커널을 사용하고, decode_split_cta를 seq_len에
}

// (4) 레지스터 스푼링 방지
fact register_pressure_limit(t: TracelItem.Compile) {
    assert: "cuTile 생성 커널의 레지스터 사용량이 255를 초과하면 로컬 메모리 스푼이 발
    action: "prefill_tile_m을 64로 제한하고, fusion_policy == 'full_fusion' 시 내부 변
}

// (5) Green Contexts 적용 기준
fact green_contexts_condition(t: TracelItem.Workload) {
    assert: "동시에 처리할 Prefill 요청과 Decode 요청이 각각 1건 이상이고, 레이턴시 만
    action: "sm_partitioning을 'green_contexts'로 설정하고 prefill_sm_ratio를 Prefill
}

}

// =====
// 4. 에이전트 루프 (아키텍트)
// =====
agent_loop nano_vllm_Architect {
    step "작업 부하 분석" {
        llm.query {
            prompt: "입력으로 주어질 전형적인 시퀀스 길이, 배치 크기, 동시 요청 수를 분석하
            output_var: "avg_seq_len, max_seq_len, batch_size, concurrent_requests"
        }
    }

    step "추론 페이즈별 파이프라인 결정" {
        // Prefill & Decode 파이프라인 독립 구성
        for each phase in ["prefill", "decode"] {
            apply {
                inference_phase: phase,
                execution_mode: "cuda_graph", // 강제 (지식 1)
                fusion_policy: "full_fusion", // 강제 (지식 2,4)
                kv_cache_mode: "contiguous" // 초기 버전은 단순하게
            }
        }
    }

    step "SM 파티셔닝 검토" {

```

```

    if (concurrent_requests > 1) {
        apply { sm_partitioning: "green_contexts", prefill_sm_ratio: 0.75 }
    } else {
        apply { sm_partitioning: "none" }
    }
}

step "커널 튜닝 루프 호출" {
    tool.engineering_loop { name: "cuTile_Tuner" }
}

// =====
// 5. 엔지니어링 루프 (튜너)
// =====
engineering_loop cuTile_Tuner {
    // Prefill 탐색 공간
    parameter p_tile_m: [64, 128]
    parameter p_tile_n: [64, 128]

    // Decode 탐색 공간
    parameter d_kv_block: [16, 32]
    parameter d_split_cta: [1, 2, 4]

    measure {
        cmd: "python bench_nano_vllm.py --phase {{inference_phase}} --pm {{p_tile_m}} --pn {{p_tile_n}} --d_kv_block {{d_kv_block}} --d_split_cta {{d_split_cta}}"
        metric: "tokens_per_second"
        objective: "maximize"
    }

    constraint {
        // 레지스터 압박 제한 (지식 4)
        assert: "cuTile 생성 커널의 register count <= 255"
        // CUDA Graph 최대 시퀀스 길이 제약
        assert: "max_seq_len >= benchmark의 최대 시퀀스 길이"
    }
}

// =====
// 6. 빌드 (최종 산출물)
// =====
build {
    artifact "nano_vllm_cutile_fused_kernels.py" {
        generator: "python compile_fused_kernels.py --prefill_tile_m {{p_tile_m}} ..."
    }
    artifact "nano_vllm_cuda_graph_runner.py" {
        generator: "python generate_graph_runner.py --num_layers 12 --max_seq_len {{max_seq_len}}"
    }
    artifact "green_context_config.json" {
        condition: "sm_partitioning == 'green_contexts'"
        content: "SM allocation plan"
    }
}
```